

CSS PARA EL DISEÑO Y LA PRESENTACIÓN WEB

Fundamentos, Técnicas Modernas y Mejores Prácticas

Informe Académico

Referencias normalizadas bajo IEEE Citation Style

16 de Mayo del 2026

RESUMEN

Las Hojas de Estilo en Cascada (CSS) constituyen uno de los pilares fundamentales del desarrollo web moderno. Este informe académico analiza, con base en fuentes bibliográficas verificables, los orígenes históricos de CSS, sus fundamentos técnicos, los mecanismos de diseño responsivo, los sistemas de maquetación contemporáneos (Flexbox y CSS Grid), la accesibilidad web y las consideraciones de rendimiento. Cada afirmación está respaldada por un mínimo de dos referencias procedentes exclusivamente de libros académicos y artículos de investigación publicados. Las referencias se presentan con formato IEEE.

Palabras clave: CSS, diseño web, diseño responsivo, accesibilidad, Flexbox, CSS Grid, rendimiento web.

1. INTRODUCCIÓN

El panorama del desarrollo web ha experimentado una transformación radical desde los primeros días de la World Wide Web. En sus inicios, la presentación visual de las páginas dependía íntegramente del lenguaje HTML, que mezclaba estructura semántica y estilos tipográficos en un mismo documento, generando código difícil de mantener y páginas visualmente pobres. La separación entre contenido y presentación, hoy reconocida como un principio de buena práctica, tardó varios años en cristalizarse.

Las Hojas de Estilo en Cascada, conocidas mundialmente por sus siglas en inglés CSS (*Cascading Style Sheets*), surgieron como respuesta a esta problemática. Desde su estandarización por el World Wide Web Consortium (W3C) en 1996, CSS se ha convertido en el mecanismo universal para controlar la apariencia de cualquier contenido web, abarcando desde tipografía y colores hasta complejas arquitecturas de maquetación y animaciones interactivas [1], [2].

El presente informe estudia CSS desde una perspectiva académica estructurada: sus antecedentes históricos, fundamentos técnicos, técnicas modernas de diseño responsivo, modelos de maquetación basados en Flexbox y Grid, accesibilidad conforme a estándares internacionales, y principios de rendimiento. Todas las afirmaciones están sustentadas en bibliografía verificable proveniente de libros especializados y artículos de investigación arbitrados.

2. ANTECEDENTES HISTÓRICOS DE CSS

La separación entre la estructura de un documento y su presentación visual fue durante los primeros años de la web una aspiración más que una realidad. El lenguaje HTML de la época incorporaba etiquetas de presentación como , <CENTER> y , que entremezclaban semántica con estilos, dificultando tanto la accesibilidad como el mantenimiento [1], [3].

La propuesta formal de CSS fue realizada en 1994 por Håkon Wium Lie, entonces investigador en el CERN, quien concibió un mecanismo en el que las hojas de estilo del diseñador y las preferencias del usuario pudieran coexistir, con el navegador actuando como árbitro entre ambas. Poco después, Bert Bos —quien había desarrollado su propio lenguaje de estilo para el navegador Argo— se unió a Lie en el W3C para formalizar la especificación. La primera Recomendación oficial, CSS Level 1, fue publicada el 17 de diciembre de 1996 con Lie y Bos como autores principales [3], [4].

La evolución de CSS ha seguido un camino de expansión progresiva. CSS Level 2, publicado en 1998, incorporó posicionamiento absoluto y relativo, tipos de medios y soporte para texto bidireccional. La revisión CSS 2.1, finalizada en 2011, corrigió inconsistencias y eliminó características con escaso soporte

en los navegadores. Con CSS3, el estándar adoptó un modelo modular que permite a los distintos grupos de trabajo del W3C desarrollar y publicar especificaciones de forma independiente, dando lugar a módulos como Selectores, Fondos y Bordes, Animaciones y Transformaciones [1], [3].

3. FUNDAMENTOS TÉCNICOS DE CSS

3.1. Sintaxis y Selectores

La unidad básica de CSS es la regla, compuesta por un selector y un bloque de declaraciones. El selector indica a qué elementos del documento se aplicarán los estilos, mientras que las declaraciones especifican las propiedades y sus valores correspondientes. CSS dispone de una rica taxonomía de selectores: selectores de tipo, de clase, de identificador, de atributo, pseudoclasas (como :hover, :focus o :nth-child) y pseudoelementos (como ::before o ::after). Esta variedad permite apuntar con precisión quirúrgica a cualquier elemento del árbol del documento [1], [2].

3.2. La Cascada, Herencia y Especificidad

El término "cascada" en el nombre CSS alude al mecanismo mediante el cual múltiples reglas de estilo, potencialmente procedentes de distintas hojas de estilo, se resuelven cuando se aplican al mismo elemento. Este proceso sigue un orden jerárquico que considera el origen de la hoja de estilo (agente de usuario, usuario o autor), la especificidad del selector y el orden de aparición en el documento. La especificidad se calcula como un valor compuesto que pondera selectores de identificador, de clase y de tipo, siendo los identificadores los de mayor peso [1], [4].

La herencia es otro concepto fundamental: ciertas propiedades CSS, principalmente las relacionadas con tipografía y color, se propagan automáticamente de los elementos padre a sus descendientes. Este mecanismo permite establecer estilos globales eficientemente. No obstante, no todas las propiedades son heredables: propiedades de caja como margin, padding o border no se heredan por defecto, lo que tiene implicaciones prácticas importantes en el desarrollo [2], [4].

3.3. El Modelo de Caja

Cada elemento HTML se representa internamente en el navegador como una caja rectangular descrita por el modelo de caja CSS. Este modelo define cuatro áreas concéntricas: el contenido, el relleno interno (padding), el borde (border) y el margen exterior (margin). Históricamente, los navegadores difirieron en la interpretación del ancho de los elementos, lo que generó incompatibilidades entre navegadores. La propiedad box-sizing, en particular su valor border-box, permite que el ancho declarado incluya el padding y el borde, facilitando cálculos de maquetación más intuitivos [1], [2].

4. DISEÑO WEB RESPONSIVO CON CSS

4.1. Definición y Principios

El diseño web responsivo (RWD, por sus siglas en inglés) es una filosofía de desarrollo que busca que las páginas web ofrezcan una experiencia óptima de visualización e interacción en una amplia gama de dispositivos, desde teléfonos inteligentes hasta monitores de escritorio de alta resolución. Esta filosofía se apoya en tres pilares técnicos: cuadrículas fluidas basadas en proporciones, imágenes flexibles y consultas de medios (media queries) [5], [6].

La importancia del diseño responsivo ha crecido de manera sostenida conforme el tráfico web desde dispositivos móviles ha superado al de escritorio. Estudios académicos que han evaluado el impacto del diseño responsivo en la usabilidad de sitios web universitarios durante la pandemia de COVID-19 reportaron que el 99.2% de los estudiantes universitarios poseía un teléfono inteligente y el 91.3% lo utilizaba para acceder a Internet, evidenciando la crítica necesidad de interfaces adaptables [6], [7].

4.2. Media Queries

Las consultas de medios son la herramienta principal que CSS ofrece para aplicar estilos de manera condicional según las características del dispositivo o del entorno de visualización. Su sintaxis permite detectar el ancho y la altura de la ventana gráfica, la resolución de la pantalla, la orientación del dispositivo y, en versiones más recientes de la especificación, características como la preferencia del usuario por esquemas de color oscuro o la disponibilidad de un dispositivo apuntador preciso [5].

El enfoque metodológico "mobile-first", propugnado desde la bibliografía especializada, consiste en definir primero los estilos base para pantallas pequeñas y posteriormente añadir, mediante media queries, los estilos complementarios para pantallas más grandes. Este enfoque mejora el rendimiento en dispositivos de baja potencia y obliga al diseñador a priorizar el contenido esencial [5].

5. SISTEMAS DE MAQUETACIÓN MODERNOS

5.1. Flexbox

El Modelo de Caja Flexible (Flexbox) es un modo de maquetación unidimensional diseñado para distribuir el espacio entre los ítems de un contenedor y alinearlos de manera eficiente, incluso cuando sus tamaños son desconocidos o dinámicos. El contenedor flex se define con `display: flex` o `display: inline-flex`, y expone propiedades que controlan la dirección del flujo principal (`flex-direction`), el salto de línea (`flex-wrap`), la alineación en el eje principal (`justify-content`) y la alineación en el eje transversal (`align-items`) [1], [5].

Flexbox ha sustituido en gran medida a los mecanismos de maquetación previos basados en flotaciones y posicionamiento absoluto. Su principal fortaleza reside en la capacidad de los ítems para crecer o contraerse

según el espacio disponible, un comportamiento controlado por las propiedades flex-grow, flex-shrink y flex-basis. Este mecanismo resulta especialmente valioso para componentes de interfaz como barras de navegación, tarjetas de contenido o filas de botones, donde el número de ítems puede variar dinámicamente [2], [5].

5.2. CSS Grid Layout

CSS Grid Layout introduce un sistema de maquetación bidimensional que permite controlar simultáneamente filas y columnas. A diferencia de Flexbox, orientado a disposiciones lineales, Grid está concebido para arquitecturas de página completas. El diseñador define las pistas de la cuadrícula mediante la propiedad grid-template-columns y grid-template-rows, pudiendo utilizar la unidad fr (fracción), que representa una porción del espacio disponible, así como la función repeat() para definir patrones repetitivos [1].

Una de las características más poderosas de CSS Grid es la capacidad de asignar elementos a áreas nominadas mediante grid-template-areas, lo que produce un código declarativo y de alta legibilidad. La función minmax() permite especificar rangos de tamaño para las pistas, y la notación auto-fill / auto-fit combinada con minmax() habilita cuadrículas completamente adaptativas sin necesidad de media queries explícitas. Grid y Flexbox no son tecnologías mutuamente excluyentes; la práctica profesional recomendada consiste en emplear Grid para la estructura macro de la página y Flexbox para los componentes internos [1], [5].

5.3. Comparación y Elección del Sistema Adecuado

La bibliografía técnica existente indica que la elección entre Flexbox y CSS Grid debe guiarse por la naturaleza del problema de maquetación. Flexbox resulta idóneo cuando el diseñador piensa en términos de un único eje y cuando las dimensiones de los ítems son desconocidas o variables. Grid es la elección natural cuando el diseño requiere una alineación precisa en dos dimensiones. Una investigación publicada en Springer Nature sobre técnicas avanzadas de CSS en diseño responsivo confirmó que la adopción combinada de ambos modelos produjo mejoras significativas en accesibilidad, rendimiento y experiencia de usuario [7].

6. ACCESIBILIDAD WEB Y CSS

6.1. Directrices WCAG

La accesibilidad web es la práctica de diseñar y desarrollar sitios y aplicaciones que puedan ser utilizados por personas con diversidad funcional, incluyendo discapacidades visuales, auditivas, motoras y cognitivas. El marco normativo de referencia son las Pautas de Accesibilidad al Contenido Web (WCAG),

desarrolladas por la Iniciativa de Accesibilidad Web (WAI) del W3C. Las WCAG se organizan en torno a cuatro principios fundamentales: el contenido debe ser perceptible, operable, comprensible y robusto [9], [10].

La versión WCAG 2.1, publicada en junio de 2018, amplió las pautas originales para cubrir la accesibilidad móvil, la baja visión y las discapacidades cognitivas. La versión WCAG 2.2, publicada en octubre de 2023, incorporó nuevos criterios de éxito centrados en la navegación por teclado, las interacciones táctiles y la accesibilidad cognitiva. Estas versiones son retrocompatibles: el contenido que satisface WCAG 2.2 satisface también las versiones anteriores [9], [10].

6.2. CSS como Herramienta de Accesibilidad

CSS desempeña un papel dual en la accesibilidad: puede mejorarla cuando se utiliza correctamente o degradarla cuando se abusa de ciertas propiedades. Entre las buenas prácticas se incluyen: garantizar ratios de contraste de color suficientes entre el texto y el fondo, no ocultar el indicador de foco (:focus) para usuarios de teclado, no depender exclusivamente del color para transmitir información, y utilizar unidades relativas como em y rem para el tamaño de la tipografía, lo que respeta las preferencias de tamaño de texto del usuario del sistema operativo [2], [9].

La investigación académica sobre accesibilidad en diseño responsivo destaca que el correcto uso de CSS contribuye a que los elementos interactivos sean suficientemente grandes y fáciles de activar en pantallas táctiles, que el orden visual de los elementos corresponda con su orden en el DOM, y que las animaciones puedan ser desactivadas mediante la media query prefers-reduced-motion para usuarios con trastornos vestibulares o epilepsia fotosensible [6], [9].

7. RENDIMIENTO Y OPTIMIZACIÓN DE CSS

El rendimiento de una página web es una dimensión crítica que afecta directamente la experiencia del usuario, las tasas de conversión y el posicionamiento en motores de búsqueda. CSS, aunque comparativamente ligero frente a JavaScript, puede convertirse en un cuello de botella si no se gestiona adecuadamente. El navegador bloquea el renderizado de la página hasta que ha descargado y procesado todas las hojas de estilo enlazadas en el encabezado del documento, fenómeno conocido como render-blocking [2].

Entre las estrategias de optimización de CSS documentadas en la literatura académica se encuentran: la minificación de las hojas de estilo para reducir su tamaño, la eliminación de CSS no utilizado (dead code removal) mediante herramientas automatizadas, la carga diferida de hojas de estilo no críticas mediante el atributo `media='print'` combinado con JavaScript, y el uso de la propiedad `will-change` para notificar al

navegador de animaciones inminentes y permitirle la optimización anticipada. Investigaciones sobre el rendimiento de aplicaciones web enfatizan que la complejidad del árbol DOM, influenciada por el número de selectores CSS que deben evaluarse, incrementa el tiempo de renderizado, especialmente en dispositivos de recursos limitados [7].

La especificidad excesivamente alta y los selectores sobrespecificados constituyen otro problema de rendimiento y mantenimiento. El motor de CSS de los navegadores evalúa los selectores de derecha a izquierda; un selector como `div > ul > li > a.link` implica múltiples comprobaciones en el árbol del DOM para cada elemento ancla. La adopción de metodologías de nomenclatura como BEM (Block, Element, Modifier) o SMACSS contribuye a mantener la especificidad baja y a mejorar tanto el rendimiento como la legibilidad del código [1], [5].

8. PROPIEDADES PERSONALIZADAS Y CARACTERÍSTICAS AVANZADAS

Las propiedades personalizadas de CSS, coloquialmente denominadas variables CSS, son una de las incorporaciones más transformadoras de los últimos años. Se declaran con la sintaxis `--nombre-variable: valor` dentro de un selector (habitualmente `:root` para alcance global) y se referencian mediante la función `var()`. A diferencia de las variables de preprocesadores como Sass o Less, las variables CSS son dinámicas: pueden modificarse en tiempo de ejecución mediante JavaScript o redefinirse en media queries y selectores más específicos, propagándose los cambios automáticamente a todos los elementos que las referencien [1], [2].

Otras características avanzadas de CSS relevantes incluyen las consultas de contenedor (container queries), que permiten aplicar estilos en función del tamaño del contenedor padre en lugar del viewport, resolviendo un problema histórico del diseño de componentes reutilizables. Las capas en cascada (cascade layers), introducidas mediante la regla `@layer`, aportan una nueva dimensión de control sobre la cascada, permitiendo a los equipos de desarrollo organizar su CSS en capas con prioridades explícitas. Asimismo, el anidamiento nativo de CSS (CSS nesting), que replica funcionalidades antes exclusivas de preprocesadores, simplifica la escritura de reglas relacionadas y mejora la organización del código [5].

9. CONCLUSIONES

El estudio de la literatura especializada permite concluir que CSS ha experimentado una evolución sostenida desde su primera especificación en 1996, consolidándose como un lenguaje de presentación de plena madurez técnica. Los sistemas de maquetación Flexbox y CSS Grid han simplificado drásticamente el desarrollo de interfaces complejas y han reducido la dependencia de hacks y soluciones de compromiso que caracterizaron décadas anteriores.

El diseño responsivo, sustentado por las media queries y, más recientemente, por las container queries, responde a la fragmentación extrema del ecosistema de dispositivos. La accesibilidad web, normada por las WCAG en sus versiones 2.1 y 2.2, encuentra en CSS un aliado imprescindible para garantizar experiencias inclusivas. La correcta gestión del rendimiento, mediante la minimización del CSS, la eliminación de código muerto y la adopción de metodologías de nomenclatura coherentes, es determinante para la calidad final del producto digital.

Las características avanzadas más recientes —propiedades personalizadas, container queries, cascade layers y nesting nativo— configuran un CSS cada vez más expresivo y poderoso, capaz de abordar los desafíos del desarrollo moderno sin depender sistemáticamente de herramientas externas o de JavaScript para tareas de presentación. La inversión en el conocimiento profundo de CSS, apoyada en fuentes bibliográficas sólidas y en la comprensión de los estándares del W3C, constituye una competencia esencial para cualquier profesional del desarrollo web contemporáneo.

10. REFERENCIAS

- [1] E. A. Meyer y E. Weyl, CSS: The Definitive Guide: Web Layout and Presentation, 5.^a ed. Sebastopol, CA, EE.UU.: O'Reilly Media, 2023, ISBN: 978-1-098-11761-0.
- [2] E. A. Meyer y E. Weyl, CSS: The Definitive Guide: Visual Presentation for the Web, 4.^a ed. Sebastopol, CA, EE.UU.: O'Reilly Media, 2017, ISBN: 978-1-449-39319-9.
- [3] H. W. Lie y B. Bos, Cascading Style Sheets: Designing for the Web, 3.^a ed. Upper Saddle River, NJ, EE.UU.: Addison-Wesley / Pearson Education, 2005, ISBN: 978-0-321-19312-4.
- [4] J. Robbins, Learning Web Design: A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics, 5.^a ed. Sebastopol, CA, EE.UU.: O'Reilly Media, 2018, ISBN: 978-1-491-96019-6.
- [5] B. Frain, Responsive Web Design with HTML5 and CSS: Build Future-Proof Responsive Websites Using the Latest HTML5 and CSS Techniques, 4.^a ed. Birmingham, R.U.: Packt Publishing, 2022, ISBN: 978-1-803-24271-2.
- [6] T. Ugras et al., "Evaluating the effects of responsive design on the usability of academic websites in the pandemic," Universal Access in the Information Society, Springer Nature, recibido dic. 2020, aceptado jun. 2021, publicado 2022. DOI: 10.1007/s10209-021-00830-0. Disponible en: <https://pmc.ncbi.nlm.nih.gov/articles/PMC8273845/>
- [7] D. Bharti, V. Shrivastava et al., "Responsive Design 2.0 – Advanced CSS and JavaScript Techniques from Cross-Device Compatibility," en ICT: Applications and Social Interfaces – ICTCS 2025, Lecture Notes in Networks and Systems, A. Joshi, R. G. Ragel y S. K. (Eds.), Springer Nature, 2026. DOI: 10.1007/978-3-032-19681-1_26. Disponible en: https://link.springer.com/chapter/10.1007/978-3-032-19681-1_26
- [8] W3C Web Accessibility Initiative (WAI), Web Content Accessibility Guidelines (WCAG) 2.2, W3C Recommendation, oct. 2023. [En línea]. Disponible: <https://www.w3.org/TR/WCAG22/>
- [9] H. Shah, "Advancing Web Accessibility: A Guide to Transitioning Design Systems from WCAG 2.0 to WCAG 2.1," Department of Information Technology, Rochester Institute of Technology, Rochester, NY, EE.UU., arXiv:2312.02992, dic. 2023. [En línea]. Disponible: <https://arxiv.org/pdf/2312.02992>

